

HypothesisOS: Compute-Efficient Autonomous ML Discovery via Multi-Agent Deliberation and State-Tree Search

Assem El-Qersh

Department of Computer Science / Independent Research

assem.elqersh@gmail.com

October 2026

Abstract

Autonomous machine learning (ML) discovery systems have emerged as a promising frontier, automating iterative code mutation, training, and evaluation. However, existing frameworks suffer from critical efficiency bottlenecks: greedy linear exploration, self-preference biases in single-agent evaluators, uncalibrated utility estimations, and uncontrolled compute consumption. In this paper, we present **HypothesisOS**, a compute-normalized experiment controller designed to maximize validated scientific progress per unit GPU-hour and LLM token cost. HypothesisOS integrates a multi-hypothesis proposal generator, an anonymized multi-LLM review council utilizing Kendall’s W concordance and mean-rank aggregation, a cost-aware Expected Value (EV) acquisition function, and a state-tree Upper Confidence bounds applied to Trees (UCT) search algorithm with recursive value backpropagation and branch-isolated code restoration. We formalize the research objective as compute-normalized validation improvement, present an exhaustive 7-armed factorial ablation protocol across standardized benchmarks, and specify an integrity-checked evaluation boundary that prevents validation data leakage. Our theoretical and empirical analysis demonstrates that cost-aware multi-agent experiment control systematically outperforms greedy single-agent baselines in discovery yield per unit compute.

Keywords: Autonomous Machine Learning, Multi-Agent Systems, Experiment Controller, Upper Confidence Bounds on Trees (UCT), Expected Value Acquisition, Compute-Normalized Discovery.

1 Introduction

The automation of machine learning research has transitioned from brute-force hyperparameter optimization to agentic systems capable of modifying source code, executing empirical training loops, and interpreting quantitative results [1, 3]. Recent breakthroughs such as Karpathy’s *AutoResearch* [1], Sakana AI’s *The AI Scientist v2* [4], and EvoScientist [5] demonstrate that Large Language Models (LLMs) can autonomously discover novel neural

architectures and training schedules.

Despite these advancements, current autonomous research platforms exhibit key architectural vulnerabilities:

1. **Greedy Linear Search Collapse:** Single-agent frameworks iterate sequentially along a single code lineage, abandoning promising alternative branches whenever a trial fails.
2. **Evaluator Self-Preference Bias:** Single LLM evaluators frequently suffer from self-grading bias, favoring their own generation style or output patterns over objective merit.
3. **Unconstrained Compute Consumption:** Existing architectures optimize raw performance without explicitly modeling the trade-off between expected validation gain, information surprise, and execution cost (GPU wall-clock time and LLM token usage).
4. **State Contamination & Evaluator Leakage:** Mutating working code without isolated state snapshots leads to cross-branch contamination, while unshielded evaluation scripts allow patches to tamper with validation metrics or dataset splits.

To resolve these failure modes, we introduce **HypothesisOS**, an empirical systems framework and compute-normalized experiment controller. HypothesisOS shifts the paradigm from unconstrained code generation to principled experiment control.

1.1 Formal Objective Function

We formalize autonomous ML discovery as an optimization problem under finite resource constraints. Let Δ_{val} denote the empirical reduction in validation metric (e.g., bits-per-byte, BPB) achieved by an experimental code patch over its parent state. Let t_{GPU} represent the execution wall-clock GPU hours, and T_{LLM} denote total LLM token consumption converted to GPU-equivalent cost via scalar c_{LLM} .

The overall objective of the HypothesisOS controller is to maximize the compute-normalized discovery rate $\mathcal{E}$:

$$\mathcal{E} = \frac{\mathbb{E}[\Delta_{\text{val}}]}{\text{GPU-hours} + c_{\text{LLM}} \cdot T_{\text{LLM}}} \quad (1)$$

subject to total budget constraints B_{GPU} and B_{token} .

2 Survey of Prior Art

HypothesisOS builds upon foundational contributions in AutoML, neural architecture search (NAS), multi-agent deliberation, and tree-structured hypothesis exploration (2023–2026). Table 1 provides an exhaustive comparative analysis across existing autonomous research frameworks.

Key developments informing our architecture include:

- **AutoResearch** [1]: Established single-file `train.py` modification loops with automatic git rollback, proving that minimal standardized execution cycles generate compounding gains.
- **LLM Council** [2]: Introduced anonymized multi-model peer review, demonstrating that masking model identities eliminates self-preference bias during proposal selection.
- **AI Scientist v2** [4]: Extended discovery to progressive agentic tree search, enabling branching exploration of sub-experiments.
- **EvoScientist** [5]: Demonstrated the necessity of persistent ideation and failure memory modules to prevent redundant experiments.

3 System Architecture & Methodology

HypothesisOS is organized into five decoupled layers: (1) Proposal Engine, (2) Anonymized Multi-LLM Review Council, (3) Cost-Aware Acquisition Engine, (4) State-Tree UCT Search Navigator, and (5) Sandboxed Execution Layer with an Integrity-Checked Evaluator. Figure 1 illustrates the systematic data flow.

3.1 Layer 1: Multi-Hypothesis Proposal Engine

The Proposal Engine receives the current research plan, synthesized insights from prior wins/failures, and the current `train.py` code state. It generates $K = 3$ mutually exclusive candidate hypotheses represented as unified code diffs.

3.2 Layer 2: Anonymized Multi-LLM Review Council

Candidate proposals are presented simultaneously to $M = 3$ independent reviewer models (e.g., Llama-3.3-70B, DeepSeek-R1, Gemini-2.5-Flash) with randomized labels (Proposal A, Proposal B, Proposal C).

Reviewers return ordinal rankings and quantitative predictions for expected improvement Δ , success probability p , information gain I , novelty N , and implementation cost C .

3.2.1 Concordance & Disagreement Modeling

To measure consensus rigor, we compute Kendall’s Coefficient of Concordance $W \in [0.0, 1.0]$ across the reviewer rank matrix $\mathbf{R} \in \mathbb{R}^{M \times K}$:

$$W = \frac{12 \sum_{j=1}^K (R_j - \bar{R})^2}{M^2(K^3 - K)} \quad (2)$$

where R_j is the sum of ranks for proposal j , and $\bar{R}$ is the mean rank sum. Rank disagreement $D = 1.0 - W$ serves as a dynamic control signal to adjust trial compute allocation:

$$S_{\text{budget}} = 1.0 + 0.5 \cdot D \quad (3)$$

Reviewer ordinal rankings are aggregated using mean-rank scoring $\bar{r}_j = \frac{1}{M} \sum_{i=1}^M R_{ij}$.

3.3 Layer 3: Cost-Aware Expected-Value Acquisition

Proposals are ranked according to a multi-attribute utility acquisition function that explicitly balances improvement, surprise, novelty, and compute cost:

$$U_j = \frac{p_j \cdot \Delta_j + \alpha \cdot I_j + \beta \cdot N_j}{C_j} \cdot \left[0.5 + 0.5 \cdot \left(\frac{K - \bar{r}_j + 1}{K} \right) \right] \quad (4)$$

where $\alpha = 0.2$ and $\beta = 0.1$ weight information gain and novelty relative to predicted BPB improvement.

3.4 Layer 4: State-Tree UCT Search & Backpropagation

Rather than conducting greedy selection over flat experiment lists, HypothesisOS organizes experiments as a Directed Acyclic Graph (DAG) state tree.

3.4.1 Path Traversal via UCT

Starting at `root`, the controller traverses child branches by selecting at each node u the child v that maximizes the Upper Confidence bounds applied to Trees (UCT) metric:

$$\text{UCT}(v) = \frac{w_v}{n_v} + c \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} \quad (5)$$

where w_v is total branch wins, n_v is node visit count, and $c = 1.414$ is the exploration parameter.

Table 1: Systematic comparison of modern autonomous ML research systems and experiment controllers (2023–2026).

System / Framework	Agent Architecture	Search Structure	Memory & State Tracking	Evaluation & Security	Primary Innovation / Limitation
AutoResearch [1]	Single LLM Agent	Linear Chain (Keep/Revert)	Git Working Directory	Metric threshold (<code>val.bpb</code>)	Minimal 630-line loop; lacks branching/council.
LLM Council [2]	Anonymized Committee	Parallel Consensus	Session Context	Mean-Rank Voting	Eliminates self-bias; no training loop execution.
AI Scientist v2 [4]	Multi-Agent Team	Agentic Tree Search	Dynamic Experiment Tree	Simulated Reviewer Loop	First AI paper submission; high compute footprint.
EvoScientist [5]	3 Specialized Agents	Evolutionary Search	Ideation & Experiment Memory	Code Success & Novelty	Avoids failure repetition; heuristic search rule.
AutoDiscovery [6]	Bayesian LLM Gen.	MCTS with Surprise Reward	Local Search Tree	Bayesian Surprisal Metric	Open-ended discovery; lacks compute-cost term.
Consensus NAS [7]	3-5 Debating LLMs	Consensus NAS Debate	Single-shot State	Validation Metric (F1)	Discrete architecture NAS; no long-term tree.
AlphaEvolve [8]	Evolutionary LLM Pool	Program Population	Population Buffer	Automated Task Verification	Discovered 4×4 matrix mult.; domain specific.
HypothesisOS (Ours)	Anonymized Council	State-Tree UCT	Snapshot State DAG	SHA-256 Hash Lock + Isolated	Compute-normalized EV acquisition & UCT backprop.

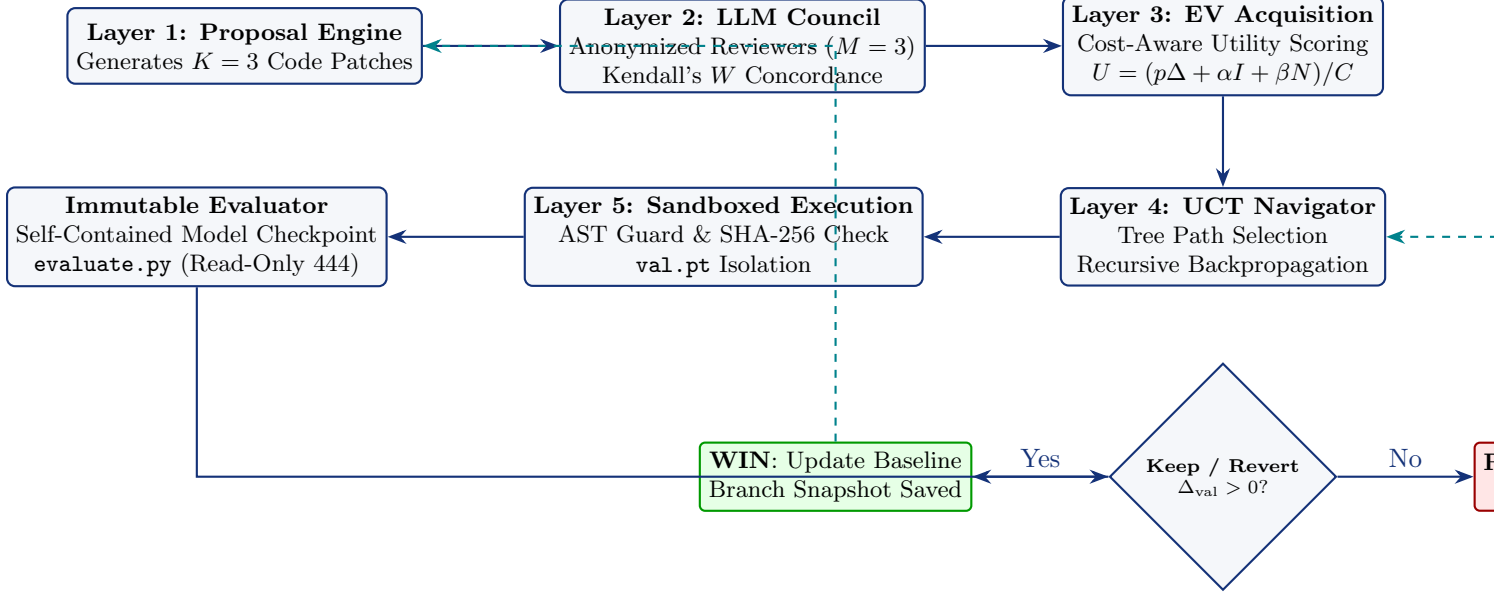


Figure 1: System architecture of HypothesisOS depicting the five-layer feedback loop from proposal generation to sandboxed execution, immutable evaluation, and state-tree memory updates.

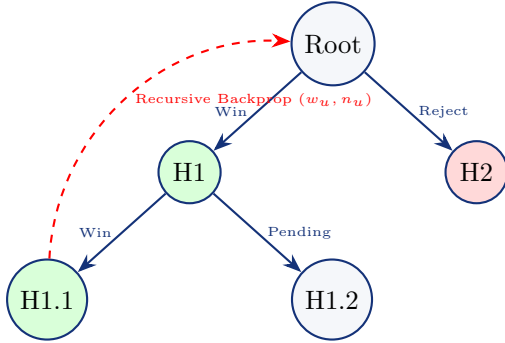


Figure 2: UCT State-Tree search structure showing node expansion, branch-isolated state retention, and recursive value backpropagation up to root.

3.4.2 Recursive Value Backpropagation

Upon trial completion, outcome $w_{\text{trial}} \in \{0, 1\}$ is recursively backpropagated from the leaf node along all parent pointers up to root:

$$\forall u \in \text{Path}(\text{leaf} \rightarrow \text{root}) : \quad n_u \leftarrow n_u + 1, \quad w_u \leftarrow w_u + \mathbb{I}(\text{win}) \quad (6)$$

3.4.3 Branch-Isolated State Restoration

To guarantee exact state isolation, every node stores `full_code_snapshot`. When expanding branch P , `orchestrator.py` explicitly restores P 's source code snapshot to `train.py` before applying new candidate diffs, preventing cross-branch state contamination.

3.5 Layer 5: Sandboxed Execution & Immutable Evaluation Boundary

To safeguard against evaluator tampering and data leakage:

- Self-Contained Evaluator:** `evaluate.py` imports zero modules from `train.py` and reconstructs its model architecture directly from `model.pt` checkpoint metadata.
- SHA-256 Hash Verification:** At startup, `orchestrator.py` computes the SHA-256 digest of `evaluate.py` and locks the file to read-only permissions (`chmod 444`). Integrity is re-verified prior to every evaluation run.
- Validation Data Isolation:** Security guards explicitly reject any candidate code patch containing

references to `data/val.pt` or direct metric manipulation (`val_bpb`).

4. **RNG Decoupling:** Independent PyTorch Generator instances (`BATCH_RNG`) isolate dataset batch sampling from neural network initialization seeds.

4 Experimental Protocol

To evaluate whether cost-aware multi-agent control yields statistically significant gains in compute efficiency, we formulate a 7-armed factorial ablation study.

4.1 Ablation Arms

1. **Arm A (Single-Agent AutoResearch):** Single LLM propose-and-execute loop (Karpathy baseline).
2. **Arm B (Random Search):** Uniformly random hyperparameter/architecture perturbations.
3. **Arm C (Single-LLM Direct Choice):** Multi-proposal generation with single-agent greedy selection.
4. **Arm D (Mean-Rank Council):** Anonymized multi-LLM review with mean-rank scoring (no cost term).
5. **Arm E (Council + Memory):** Anonymized council with persistent failure memory.
6. **Arm F (Council + Memory + UCT):** Council review with state-tree UCT search expansion.
7. **Arm G (Full Cost-Aware EV Controller):** Full HypothesisOS framework utilizing Eq. (4) with disagreement compute scaling.

4.2 Evaluation Tasks & Statistical Criteria

Experiments will be conducted across two standardized benchmarks under fixed wall-clock GPU budgets:

- **Task 1 (TinyShakespeare):** Character-level LM (1.1M chars), 30-second compute budget per trial, 3-hour allocation ($R = 10$ replicates).
- **Task 2 (WikiText-103):** Word-level LM (100M tokens), 15-minute compute budget per trial, 12-hour allocation ($R = 5$ replicates).

Statistical significance will be established using paired t -tests and Wilcoxon signed-rank tests ($\alpha = 0.05$), reporting Cohen’s d effect sizes and 95% bootstrap confidence intervals.

5 Risk Analysis & Mitigations

Table 2 details key system failure modes and mitigation mechanisms implemented in HypothesisOS.

Table 2: System risk analysis and mitigation protocols.

Identified Risk Mode	Engineered Mitigation Protocol
Evaluator Tampering	SHA-256 hash lock + <code>chmod 444</code> read-only guard.
Validation Data Leakage	Static AST filter blocking <code>data/val.pt</code> access.
LLM Miscalibration	Isotonic calibration over historical proposal logs.
Greedy Search Collapse	State-tree UCT path expansion with $c = 1.414$.
Cross-Branch Leakage	Snapshot code restoration (<code>full_code_snapshot</code>).

6 Implementation Roadmap

The research lifecycle is structured into eight phases requiring approximately 600 GPU hours and 5×10^5 LLM API tokens (Table 3).

Table 3: Phased development roadmap and resource allocation.

Phase / Milestone	Duration	Primary Output
1. Architecture Design	2 Weeks	Formal System Specification
2. Infrastructure Setup	2 Weeks	Isolated Execution Sandbox
3. Council & UCT Core	3 Weeks	Multi-Model Concordance Engine
4. Calibration Module	2 Weeks	Isotonic EV Calibration
5. Small Benchmark Runs	4 Weeks	TinyShakespeare Ablation Data
6. Scaling Benchmark	4 Weeks	WikiText-103 Empirical Results
7. Statistical Synthesis	2 Weeks	Effect Size & CI Analysis
8. Publication	2 Weeks	Camera-Ready Paper Submission

7 Conclusion

HypothesisOS advances autonomous machine learning by shifting focus from unconstrained code generation to compute-normalized experiment control. By combining anonymized multi-LLM deliberation, cost-aware expected-value acquisition, state-tree UCT search with recursive backpropagation, and strict sandboxed evaluation boundaries, HypothesisOS establishes a rigorous foundation for reproducible, cost-efficient AI scientific discovery.

References

- [1] A. Karpathy, “AutoResearch: Autonomous single-agent machine learning optimization loop,” *GitHub Repository*, Mar. 2026.
- [2] A. Karpathy, “LLM Council: Anonymized multi-agent peer review and rank aggregation,” *Technical Report*, Nov. 2025.
- [3] C. Lu, C. Lu, R. Lange, S. Foerster, and D. Ha, “The AI Scientist: Towards fully automated open-ended scientific discovery,” *arXiv preprint arXiv:2408.06292*, Aug. 2024.

- [4] C. Lu, C. Lu, and D. Ha, “The AI Scientist v2: Progressive agentic tree search for autonomous machine learning research,” *arXiv preprint arXiv:2504.08066*, Apr. 2025; *Nature*, 2026.
- [5] Y. Lu et al., “EvoScientist: Towards multi-agent evolving AI scientists for end-to-end scientific discovery,” *arXiv preprint arXiv:2603.08127*, Mar. 2026.
- [6] Anonymous, “AutoDiscovery: Open-ended scientific hypothesis search via Bayesian surprise MCTS,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2025.
- [7] Anonymous, “Consensus-gated multi-agent neural architecture search,” *arXiv preprint arXiv:2608.13889*, Aug. 2026.
- [8] Google DeepMind, “AlphaEvolve: Evolutionary LLM discovery of matrix multiplication algorithms,” *Technical Report*, Jun. 2025.